

kiVerstehen - das interaktive Notebook



Teil 1: aus Geraden werden Neuronale Netze

Wie geht's?

Klicke die Zellen mit Linksklick an und führe sie mit dem Drücken der Tasten SHIFT und ENTER oder dem "Run-Button" aus. Teste, ob alles rund läuft mit folgender Zelle.

```
In [3]: 2+2
```

```
Out[3]: 4
```

Bevor es losgeht:

Importiere die kiVerstehen-Bibliothek und wir sind startklar!

```
In [1]: import kiVerstehen
```

Aufgabe 1

In einem Tierheim finden streunende Katzen und Hunde ein vorübergehendes Zuhause. Die Tiere werden beim Empfang gewogen und gemessen. Zudem wird vermerkt, ob es sich bei den Tieren um Hund oder Katze handelt. Um Zeit und Geld zu sparen, soll das Kategorisieren nun automatisiert werden. Als Entscheidungskriterium wird erstmal das Gewicht der Tiere verwendet. Doch wo genau setzt man die Entscheidungsgrenze?

- Teste die Kategorisierung der Katzen und Hunde anhand des Gewichts. Führe dafür folgende Code-Zelle aus. Falsch kategorisierte Tiere werden grau dargestellt. Bei welchem Gewicht würdest du die Entscheidungsgrenze setzen?

```
In [2]: kiVerstehen.hundOderKatzeAnhandGewicht()
```

```
interactive(children=(FloatSlider(value=35.57707193831535, description='y_achsenabschnitt', max=50.0, min=10.0...
```

Aufgabe 2

Die Kategorisierung mithilfe des Gewichts ist dem Tierheim zu ungenau. Deshalb sollen die Tiere ab sofort mit einer Gerade ($y = \text{Steigung} * x + y\text{-Achsenabschnitt}$) kategorisiert werden.

- Führe folgende Zelle aus und teste das neue Verfahren. Findest du eine Lösung, bei der nur eine Katze und ein Hund falsch kategorisiert werden?

```
In [3]: kiVerstehen.hundOderKatzeMitGerade()
```

```
interactive(children=(FloatSlider(value=-0.9499784895546661, description='steigung', max=1.0, min=-1.0, step=0...
```

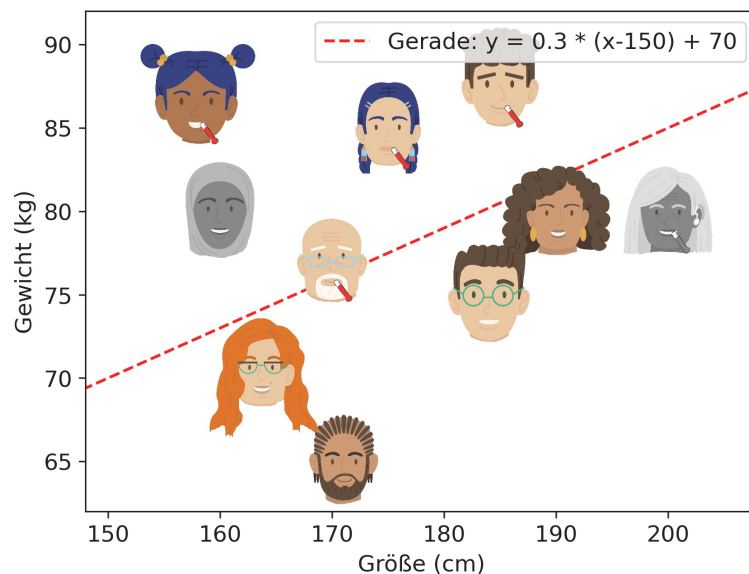
- Hältst du die Kategorisierung mithilfe der Geraden für fairer? Begründe.



Bei Katzen und Hunden ist das ja nicht so wild, wenn mal ein Tier falsch kategorisiert wird. Wenn Algorithmen über Menschen entscheiden, kann das aber sehr schwere Konsequenzen für den Einzelnen haben.

Aufgabe 3

Die GeldVorMensch-Krankenkasse hat in ihren Daten eine Korrelation zwischen Diabetes Typ 2-Erkrankten, ihrer Körpergröße und ihrem Körpergewicht entdeckt. In folgender Abbildung werden die Daten visualisiert. Personen dargestellt mit einem Fieberthermometer sind dabei an Diabetes Typ 2 erkrankt. Alle Menschen mit einem Gewicht größer als $y = 0,3 * (x - 150) + 70$ (wobei x die Körpergröße in cm ist) besitzen laut der Krankenkasse ein größeres Diabetes Typ 2-Risiko.



1. Die GeldVorMensch-Krankenkasse verlangt einen höheren Beitrag für gefährdete Personen.
2. Die MenschVorGeld-Krankenkasse hat dieselben Datensätze ausgewertet, sie bietet gefährdeten Personen kostenfreie Vorsorgeuntersuchung an und verlangt keine Zusatzbeiträge.

- Bewerte, wann es sinnvoll sein könnte, dass Maschinen Entscheidungen über Menschen treffen und wann nicht.



Aufgabe 4

Gibt es eine ideale Gerade? Das kommt darauf an, was wir mit ideal meinen.

Wir brauchen eine Möglichkeit zu beurteilen, wie ideal unsere Lösung ist. Eine erste Idee ist es, die falsch kategorisierten Tiere einfach zu zählen. Je weniger falsch kategorisierte Tiere gezählt werden, umso besser ist unsere Lösung. Die Funktion, die dies berechnet, wird **Verlustfunktion** genannt. Je kleiner der Verlust, desto besser.

- Führe folgende Zelle aus. Passe dann die Steigung und den y-Achsenabschnitt so an, dass die Verlustfunktion so klein wie möglich wird. Die Startwerte werden bei jedem Start zufällig ausgewählt.

```
In [4]: kiVerstehen.hundOderKatzeMitGeradeV1()
```

```
interactive(children=(FloatSlider(value=-0.5535785237023545, description='steigung', max=1.0, min=-1.0, step=0...
```

Eine weitere Idee ist es, die Verlustfunktion so zu verändern, dass sie die Gewichtsunterschiede der falsch kategorisierten Tiere zur Geraden misst und aufaddiert.

- Führe folgende Zelle aus. Passe dann die Steigung und den y-Achsenabschnitt so an, dass die Verlustfunktion so klein wie möglich wird. Die Startwerte werden bei jedem Start zufällig ausgewählt.

In [5]: `kiVerstehen.hundOderKatzeMitGeradeV2()`

`interactive(children=(FloatSlider(value=0.3533989748458226, description='steigung', max=1.0, min=-1.0, step=0...`

- Bewerte welche Verlustfunktion besser geeignet ist, um die Lösung zu optimieren.



Eine Verlustfunktion kann beliebig gewählt werden. Ist es uns wichtig, alle Katzen richtig zu kategorisieren, könnten wir beispielsweise die Gewichtsunterschiede der falsch kategorisierten Katzen in der Kostenfunktion stärker gewichten.

Aufgabe 5

Können die Krankenkassen auch fairere Lösungen finden, um die Menschen zu kategorisieren? Überlege dir, was fair im Kontext der jeweiligen Ansätze der Krankenkassen für dich bedeutet.

- Die GeldVorMensch-Krankenkasse will, dass Diabetes-gefährdete Personen einen höheren Beitrag bezahlen. Passe die Gerade so an, dass die Lösung möglichst fair ist.

In [6]: `kiVerstehen.krankenkassen("GeldVorMensch-Krankenkasse")`

`interactive(children=(FloatSlider(value=-0.8261223347411677, description='steigung', max=1.0, min=-1.0, step=0...`

- Die MenschVorGeld-Krankenkasse möchte gefährdeten Personen eine kostenfreie Vorsorgeuntersuchung anbieten. Passe die Gerade so an, dass die Lösung möglichst fair ist.

In [7]: `kiVerstehen.krankenkassen("MenschVorGeld-Krankenkasse")`

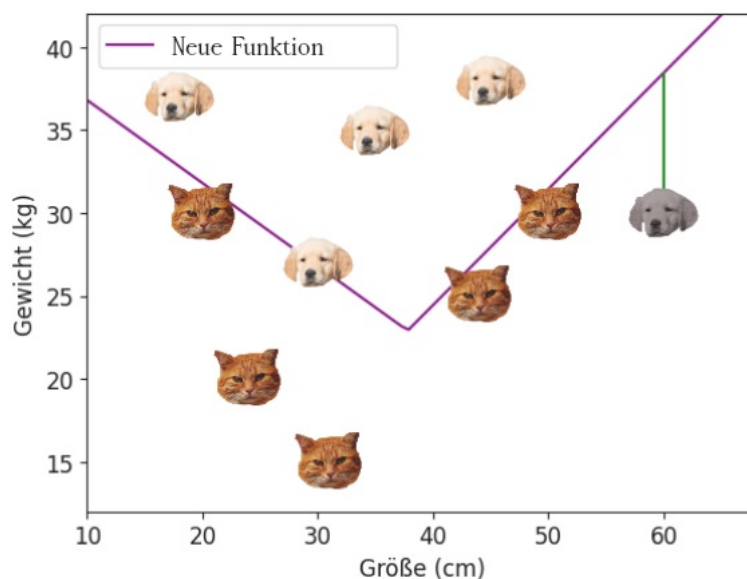
`interactive(children=(FloatSlider(value=-0.9404055611238593, description='steigung', max=1.0, min=-1.0, step=0...`

- Bewerte die Fairness deiner Lösungen.



Egal ob Mensch oder Tier, mit einer Geraden ist es schwierig alle Individuen richtig einzuordnen.

Lass es uns mit zwei Geraden versuchen!



Aufgabe 6

Was passiert wenn wir zwei Geradengleichungen aufaddieren?

- Teste folgende Zelle und begründe wieso das Aufaddieren nicht zum gewünschten Ergebnis führt.

Kleiner Hinweis:

- w_1 bezeichnet die Steigung der ersten Gerade.
- b_1 bezeichnet den y-Achsenabschnitt der ersten Gerade.
- w_2 bezeichnet die Steigung der zweiten Gerade.
- b_2 bezeichnet den y-Achsenabschnitt der zweiten Gerade.

```
In [8]: kiVerstehen.zweiGeraden()
```

```
interactive(children=(FloatSlider(value=0.02, description='w1', max=2.0, min=-2.0, step=0.05), FloatSlider(val...
```

Aufgabe 7

Wir brauchen also irgend einen Kniff, um Geradenstücke aneinanderzuschweißen.

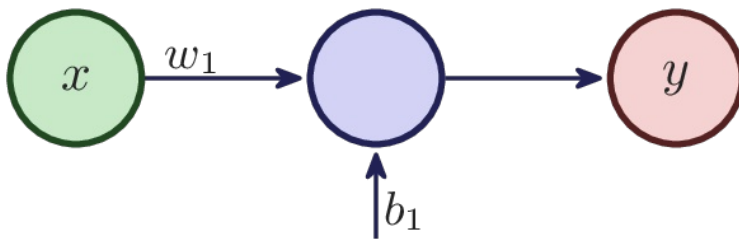
Und hier kommt das **Neuron** zum Zug. Das Übertragungsverhalten des Neurons unterscheidet sich kaum von der Geradengleichung. Der einzige Unterschied ist, dass alle Werte kleiner 0 auf 0 gesetzt werden. Die Steigung der Geradengleichung wird jetzt allerdings **Gewicht** genannt und der y-Achsenabschnitt **Bias**. Erst wenn das gewichtete Eingangssignal einen Grenzwert übersteigt (den Bias), **feuert** das Neuron. Es gibt erst dann ein Signal weiter.

- Führe folgende Zelle aus und begutachte das Übertragungsverhalten des Neurons.

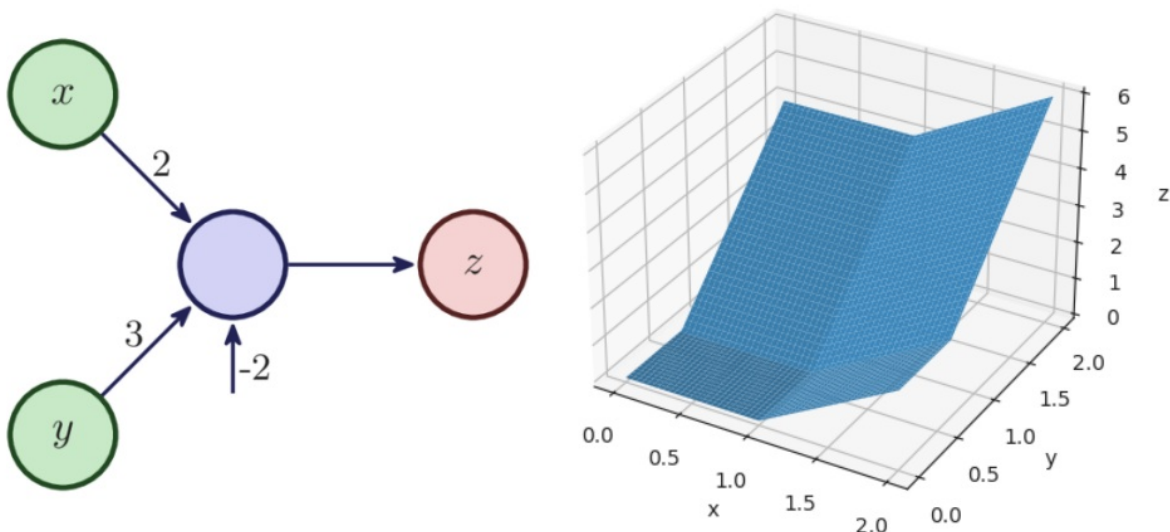
```
In [9]: kiVerstehen.einNeuron()
```

```
interactive(children=(FloatSlider(value=0.18, description='w1', max=2.0, min=-2.0, step=0.05), FloatSlider(val...
```

Schematisch dargestellt sieht das Neuron so aus. Es bekommt x als Eingabe übergeben und gibt ein y -Wert zurück.



Dem Neuron können auch mehrere Eingaben übergeben werden. Bei zwei Eingaben erhalten wir so eine dreidimensionale Übertragungsfunktion. Die Anzahl der Eingaben in ein Neuron ist nicht begrenzt. Es wird allerdings ziemlich schwer, sich die Übertragungsfunktion in so vielen Dimensionen vorzustellen.



Daher bleiben wir lieber in 2D...

Aufgabe 8

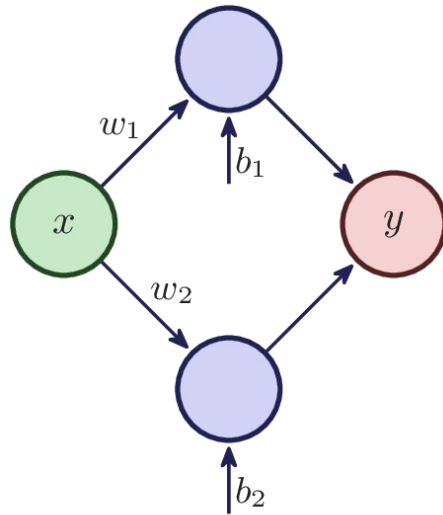
Indem wir unterschiedliche Neuronengleichungen addieren, lassen sich die Geradenstücke (2D) aneinanderschweißen. Werden Neuronenfunktionen in einem Netz miteinander summiert, spricht man von einem **Neuronalen Netz**.

- Teste folgende Zelle und beobachte, wie zwei Neuronengleichung zu der neuen Funktion addiert werden.

```
In [10]: kiVerstehen.zweiNeuronen()
```

```
interactive(children=(FloatSlider(value=0.36, description='w1', max=2.0, min=-2.0, step=0.05), FloatSlider(val...
```

Das Neuronale Netz von oben sieht schematisch dargestellt wie folgt aus. Es besteht aus zwei Neuronen, hat die Eingabe x und die Ausgabe y .



Aufgabe 9

Zurück zu unseren Katzen und Hunden. Wir verwenden jetzt unsere Neuronen, um eine genauere Lösung zu finden. Die Lösung bewerten wir wieder mit der Verlustfunktion.

- Führe die Zelle aus und passe die Gewichte w_1 und w_2 und die Biase b_1 und b_2 so an, dass du den Verlust minimierst und eine möglichst ideale Funktion findest. Wie klein kannst du den Verlust machen?

```
In [11]: kiVerstehen.hundOderKatzeZweiNeuronen()
```

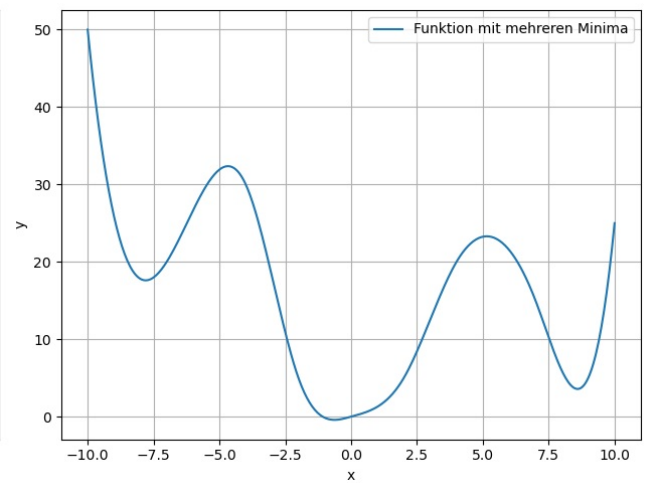
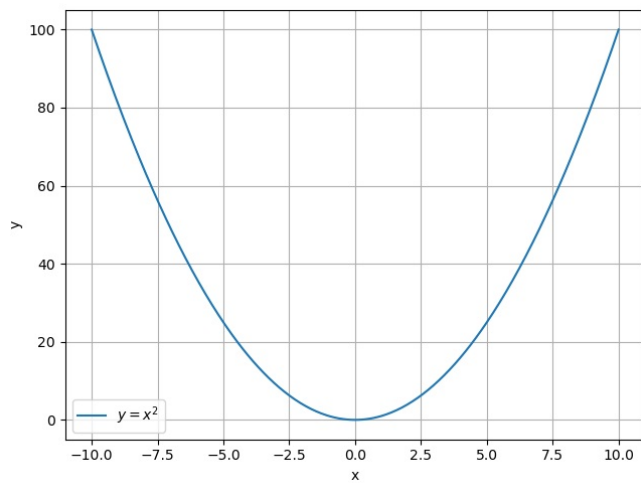
```
interactive(children=(FloatSlider(value=0.09, description='w1', max=1.0, min=-2.0, step=0.05), FloatSlider(val...
```

- Beschreibe wie du bei der Lösungsfindung vorgegangen bist.
- Wie findet man heraus, ob die eigene Lösung die beste Lösung ist? Begründe.



Du hast gerade die Parameter (Gewichte, Biase) eines kleinen Neuronalen Netzes optimiert. Das hast du wahrscheinlich gemacht, indem du die Parameter so verändert hast, dass die Verlustfunktion so klein wie möglich wird. Findet dieser Prozess automatisch statt, wird er als **Training** bezeichnet.

Wäre die Verlustfunktion (y) nur von einem Parameter (x) abhängig, könntest du dir das so vorstellen:



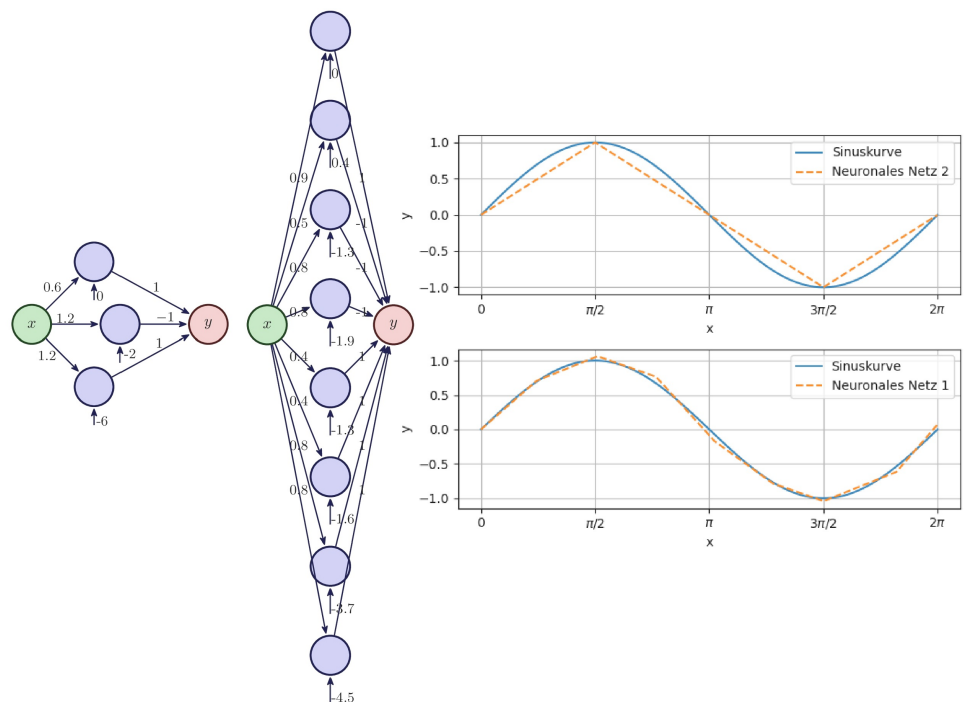
Du hast x so lange variiert bis du ein lokales Minimum gefunden hast. Ob der Verlust y beim Verändern von x steigt oder sinkt, hat dir geholfen herauszufinden, in welche Richtung du x anpassen musst. Implizit hast du also die Steigung an der Stelle x bestimmt. Der Computer geht genau so vor. Beim sogenannten **Gradientenabstieg** berechnet er die Steigung und passt die Parameter so an, dass die Verlustfunktion sich einem lokalen Minimum nähert.

Da es unterschiedliche lokale Minima gibt, kamen deine Klassenkamerad:innen eventuell zu anderen Ergebnissen. Das kannst du auf dem rechten Graphen nachvollziehen.

Aufgabe 10

Theoretisch können wir jede beliebige Funktion mit genügend Neuronen nachbauen. Denn jede Funktion lässt sich durch das Aneinanderhängen kleiner Geradenstückchen annähern. Für den Matheunterricht hat Tom eine Sinuskurve mit zwei Neuronalen Netzen angenähert. Dazu hat er einmal drei und das andere mal acht Neuronen verwendet. Dummerweise hat er vergessen, welches Schaubild zu welchem Graphen gehört.

- Ordne die Schaubilder den Graphen zu.

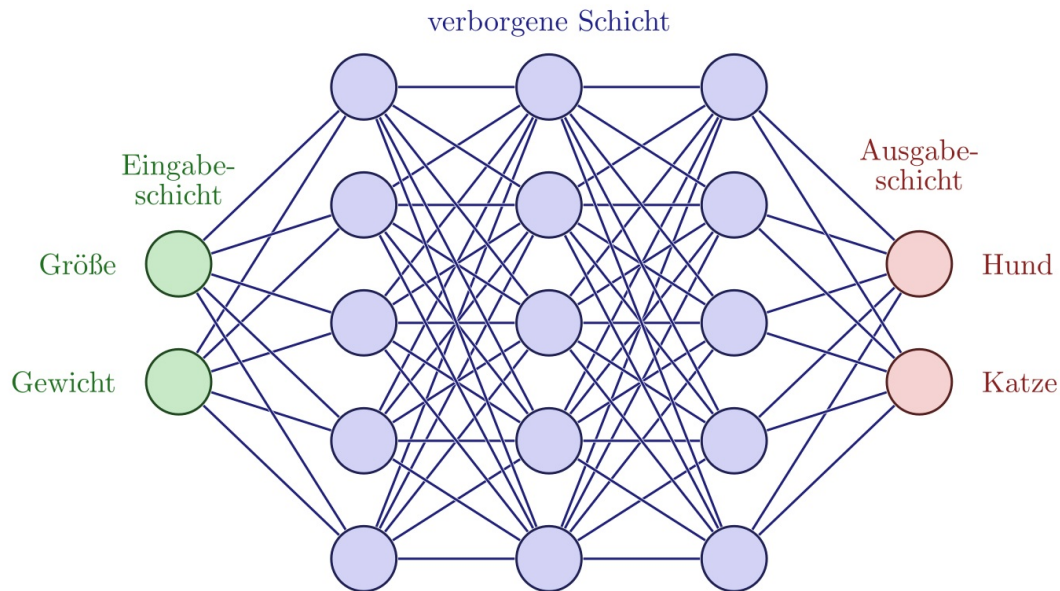


Beachte, dass Tom auch die aus den Neuronen herausführenden Signale gewichtet (mit einem Wert mal genommen) hat.

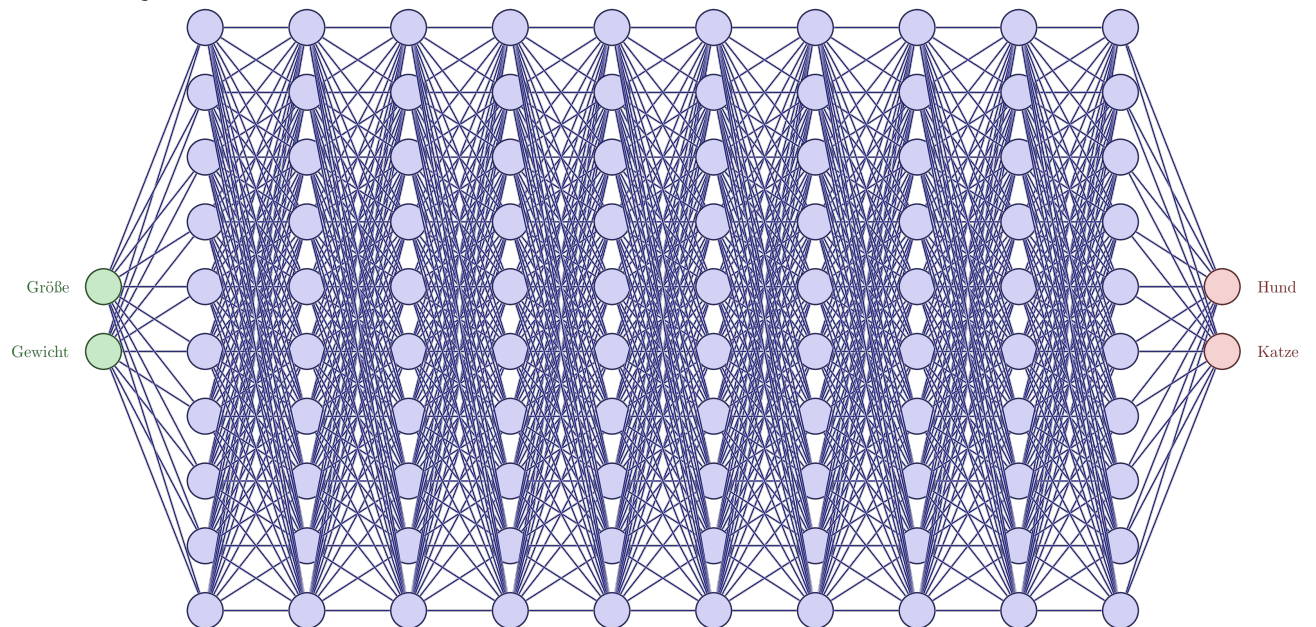
Aufgabe 11

Wie die Neuronen angeordnet sind, kann sich je nach Neuronalem Netz unterscheiden. Für das Tierheim wäre es beispielsweise sinnvoll, die Größe und das Gewicht der Tiere als Eingabe zu übergeben. Dies geschieht in der sogenannten **Eingabeschicht**. In der **Ausgabeschicht** gibt das Netz uns die Wahrscheinlichkeit, dass es sich um eine Katze oder einen Hund handelt, zurück. In der **verborgenen Schicht** sind die Neuronen in mehreren Schichten angeordnet. Alle Neuronen einer Schicht sind mit den Neuronen der nächsten Schicht verbunden. Jede Verbindung ist über ein Gewicht gewichtet. Der Bias der Neuronen wird in der Grafik nicht dargestellt. Wie du in der Grafik erkennst, gibt es jetzt sehr viele Verbindungen zwischen den Neuronen. Diese Gewichtsparameter von Hand zu

optimieren (so wie wir es bisher getan haben) wäre also sehr viel Arbeit! Und in der Realität sind Neuronale Netze noch viel viel größer.



Das Tierheim hat folgendes Neuronales Netz mit neuen Tierdaten trainiert. Es wird genutzt, um neue abgegebene Tiere in Hunde und Katzen zu kategorisieren.



- Teste das Modell und ordne die neuen Tiere den Kategorien "Hund" oder "Katze" zu.

Testdaten



neue Tiere



```
In [12]: kiVerstehen.hundOderKatzeNNtesten(größe=30, gewicht=30)
```

Vorhersage für 30 kg und 30 cm:

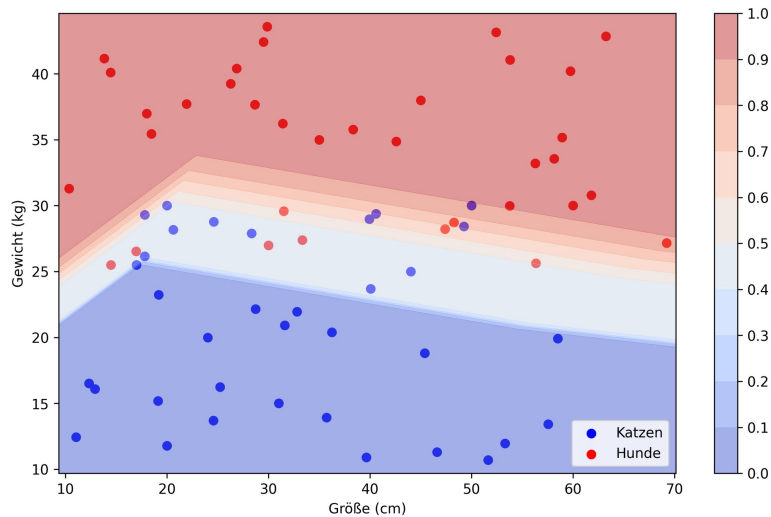
Hund: 0.609

Katze: 0.391

Zusatzaufgabe 1

Testet man das Modell auf alle Gewichts- und Größenkombinationen erhält man folgende Grafik. Die Punkte stellen die Trainingsdaten dar, die Farbwerte repräsentieren die Wahrscheinlichkeiten, dass es sich bei dem Tier um einen Hund handelt.

- Für welche Gewichts- und Größenkombinationen kategorisiert die KI neue Tiere als Katzen bzw. Hunde? Begründe.



Zusatzaufgabe 2

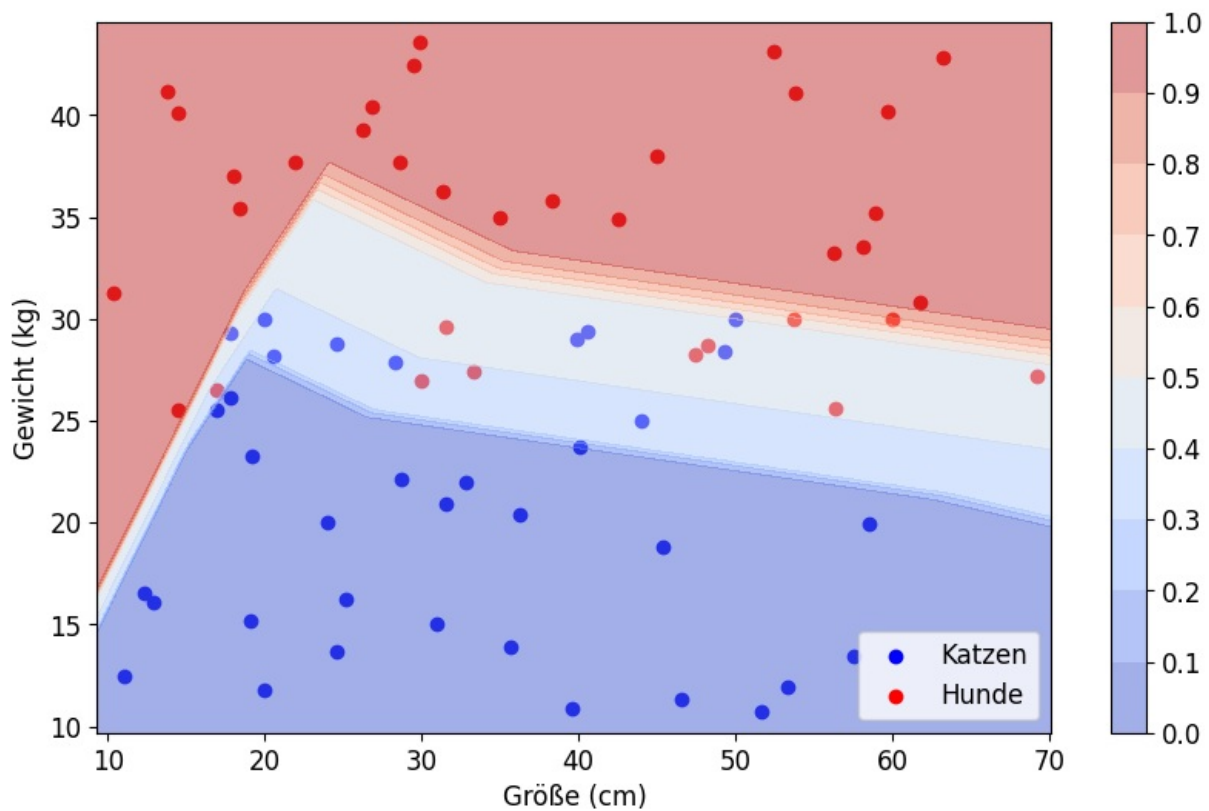
Führst du folgende Methode aus, so wird das Modell mit den Trainingsdaten trainiert. Eine Epoche ist abgeschlossen, wenn alle Trainingsdaten einmal angeschaut wurden und die Gewichte dementsprechend angepasst wurden. Um die Verlustfunktion weiter zu minimieren und ein gutes Ergebnis zu erhalten, werden einige Epochen benötigt.

- Teste einerseits, was passiert, wenn du das Modell mit der selben/unterschiedlichen Anzahl an Epochen trainierst.
- Beschreibe, was dir dabei auffällt.
- Begründe die Beobachtungen.



```
In [13]: kiVerstehen.hundOderKatzeNNtrainieren(epochen=2000)
```

```
Epoche [100/2000], Verlust: 0.3257
Epoche [200/2000], Verlust: 0.2276
Epoche [300/2000], Verlust: 0.2280
Epoche [400/2000], Verlust: 0.2257
Epoche [500/2000], Verlust: 0.2295
Epoche [600/2000], Verlust: 0.2171
Epoche [700/2000], Verlust: 0.1986
Epoche [800/2000], Verlust: 0.2084
Epoche [900/2000], Verlust: 0.2055
Epoche [1000/2000], Verlust: 0.2007
Epoche [1100/2000], Verlust: 0.2148
Epoche [1200/2000], Verlust: 0.1892
Epoche [1300/2000], Verlust: 0.1965
Epoche [1400/2000], Verlust: 0.1855
Epoche [1500/2000], Verlust: 0.2647
Epoche [1600/2000], Verlust: 0.1944
Epoche [1700/2000], Verlust: 0.2827
Epoche [1800/2000], Verlust: 0.1800
Epoche [1900/2000], Verlust: 0.1873
Epoche [2000/2000], Verlust: 0.2060
```



In diesem Beispiel konnten wir uns alle Entscheidungen der KI visualisieren. Als Eingabegrößen gab es auch nur Gewicht und Größe eines Tiers. Wie eine KI in der echten Welt auf komplexe Situationen reagiert, kann man allerdings schlecht vorhersagen. Das müsste man eigentlich testen. Das ist allerdings gar nicht so leicht, denn unsere Eingaben haben meist deutlich mehr Parameter. Alle möglichen Kombinationen durchzutesten ist daher nicht praktikabel.

In []: